UTN – FACULTAD REGIONAL BUENOS AIRES
Algoritmos y Estructura de Datos – 2026
CURSO K1043
Docente: Ing. Pablo Damián Mendez

TRABAJO PRÁCTICO GRUPAL

Marasco, Nazareno	260.293-3	nmarasco@frba.utn.edu.ar
Gutierrez Fulco, Francisco	260.213-1	fgutierrezfulco@frba.utn.edu.ar
Eugenio, Alejo	260.127-8	aeugenio@frba.utn.edu.ar
Busat Vidal, Patricio Matias	260.030-4	pbustvidal@frba.utn.edu.ar
Zacariaz, Juan Manuel	260.557-0	jzacariaz@frba.utn.edu.ar

Usuarios github

- franguti-11
- alejo-eugenio
- nmarasco
- jazacariaz00

- patriciobusat

FECHA DE PRESENTACIÓN: 07/11/2026

FECHA DE DEVOLUCIÓN: CALIFICACIÓN _____

Hipótesis de trabajo

Se consideró que la tabla está vacía cuando no tiene ninguna orden cargada y miembros siguiente x e y están inicializados con -99

Para inicializar la tabla:

Utilizamos una función que recibe una matriz del tipo orden de 200 filas por 200 columnas y lo que hace es que utilizando la función for, recorre filas y columnas, cargando en cada una de las celdas los datos iniciales de orden. Para garantizar que inicie siempre la tabla desde un estado inicial conocido.

Para cargar los archivos:

Para cargar el archivo en la tabla, primero verificamos ciertas restricciones en el archivo como la cantidad de despegues y aterrizajes o ataque kamikaze, por cada registro leído con despegue fuimos sumando a una variable contadora de despegues y lo mismo para los aterrizajes o ataques kamikazes.También utilizamos dos variables booleanas para verificar que el despegue y el aterrizaje o ataque kamikaze no estuviera acompañado de ninguna otra instrucción y que el aterrizaje o ataque kamikaze venga con su siguiente X e Y en -99.

Una vez pasada esta validación cargamos el archivo en la tabla y seguimos con el resto de las validaciones.

Primero,verificamos que en el archivo no hayan registros con el mismo X e Y para que no se pisen datos y con una variable booleana verificamos eso.

Segundo, verificamos que correspondan los siguientes x e y de cada registro, es decir que si un registro tiene un siguiente X y un siguiente Y distinto de -99 y no es un ataque kamikaze o aterrizaje,en esa celda efectivamente debe haber un registro cargado.
Tercero,pasamos a verificar que no hayan huecos dentro del trayecto del dron para este preguntamos si el siguiente X y el siguiente Y esta dentro de un rango, para el siguiente Y debe estar una columna a la izquierda, en la misma o una a la derecha con respecto a las coordenadas de la celda en la que está. Lo mismo ocurre para el siguiente X el cual puede estar una fila más arriba, en la misma o una fila más abajo con respecto a la coordina de la celda en la que está cargada la instrucción.

Por último, una vez que haya pasado por las validaciones ya mencionadas verificamos si el dron pasa de vuelta por una misma celda para esto utilizamos un tabla auxiliar booleana, la cual inicializamos todas las celdas en false.Después buscaremos las coordenadas del despegue en la tabla y se las asignaremos a una variable y dentro de un while iremos recorriendo la tabla y preguntando si esa celda en la tabla auxiliar esta en false, lo cual nos indica que no pasó, y le asignaremos como valor true. En caso de que pasemos por una celda la cual en la tabla auxiliar está en true, utilizaremos otra variable booleana para determinar que pasó por esa celda ya. En esta validación no preguntamos por el aterrizaje o el ataque kamikaze ya que nuestro trabajo se apoya en la hipótesis de que el siguiente X e Y de dicha instrucción está en -99.

En caso de que no apruebe una de las validaciones volveremos a inicializar toda la tabla.

Si se desea corregir el archivo:

Se le pregunta al usuario las coordenadas que desea cambiar, una vez ingresada, pregunta si la posición de la tabla es igual a la ingresada para ser buscada, la busca moviéndose por el archivo y si lee las coordenadas que son iguales a las pedidas en esta función se detiene ahí y pide los datos nuevamente, (coordenadas, sgte coordenada, despegue, aterrizaje, etc). De ahí se mueve para sobrescribir los datos y realizar esa misma acción. Pone la variable encontró en true para que deje de buscar.

Para crear el archivo en sí:

El usuario debe de ingresar las coordenadas donde está el dron y las coordenadas en la posición a la que quiere desplazar el dron. El resto de preguntas se responden con 1 y 0 para así ajustar los valores de verdad de los despegues, aterrizajes, ataques kamikaze, y granadas, 1 siendo el caso donde se quiere que suceda dicha instrucción y 0 cuando no se quiere que suceda. Una vez termina con estos datos se pregunta al usuario si decide seguir, poniendo 1 o 0 nuevamente, si pone 0, el proceso se repite y vuelve a la pregunta del final, en caso de poner 1 se termina el proceso de crear el archivo. (En el caso de reiteración, las coordenadas que pide deben ser iguales a las coordenadas que antes eran coordenadas siguientes.)

Caso especial de sgte: Si se desea aterrizar o hacer ataque kamikaze la coordenada siguiente deberá ponerse en -99 para que así no haya aterrizajes o un ataque kamikaze y después hayan más casillas por las que pasa el dron después de eso.

Si se desea corregir la tabla:

El usuario ingresa las coordenadas celda de la tabla que se desea modificar, entonces se pregunta si la posición “x” y “y” ingresadas existen dentro de la tabla de 200 filas por 200 columnas por lo que confirma que la posición “x” sea mayor o igual a 0 y menor a 200, a su vez confirma que la posición “y” sea mayor o igual a 0 y menor a 200. Entonces si la posición ingresada es válida pide al usuario que ingrese los nuevos elementos y los guarda en esa celda por medio de una variable auxiliar k de tipo orden.

Para guardar una tabla en un archivo:

Es una función que recibe la tabla de 200 filas por 200 columnas y la ruta en la que se desea guardar este archivo,se guarda en una variable auxiliar de tipo orden, los registros que comparte con el tipo orden archivo y abre un archivo con la ruta en write binary y escribe en las celdas que cumplan con determinadas condiciones como que los siguientes “x” y “y” son -99 o que aterrizaje es true, o que kamikaze es true o que despegue es true y entonces si alguna se cumple ahí guarda la celda en la tabla.

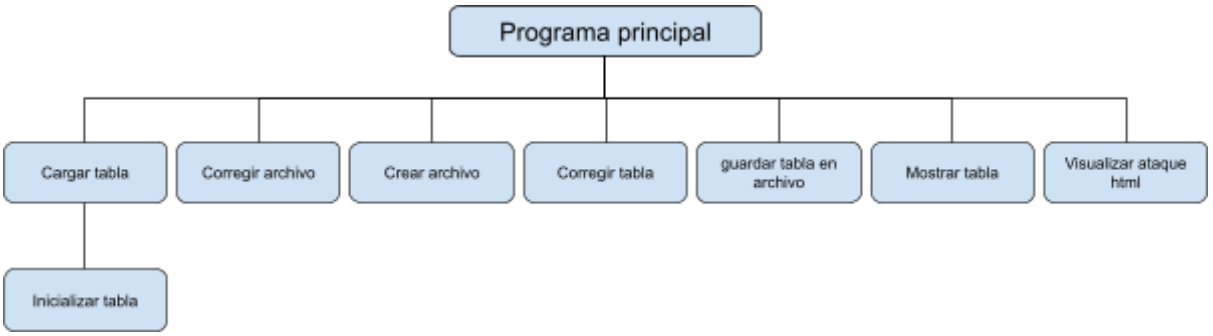
Para mostrar la tabla:

Es una función que recibe la tabla de 200 filas por 200 columnas y que usando dos variables recorre todas las celdas de la tabla, mostrando en pantalla cuál es la posición, si despego, si tiro granada1, si tiro granada 2, si aterrizo y si realizo ataque kamikaze. Además cuenta la cantidad de líneas y cuando supera las 20 pregunta si el usuario desea pasar a otra página, reiniciando el contador de cantidad de líneas si es que desea pasar de página.

Para visualizar el HTML del ataque:

Esta función nos permite por así decirlo “traducir” los datos para llevarlo a un entorno WEB, utilizamos archivos para poder escribir el documento HTML,para lograr esto utilizamos la función fprintf ya que su rol principalmente es inyectar texto estricto y estructurado al archivo abierto en disco.Tenemos el HTML con sus respectivas cajas separadas en clases para que su uso sea mucho más práctico de manejar a la hora customizar su estilo. El recorrido de esta funciona en base a 2 bucles FOR anidados, esta almacena una referencia constante a la celda actual, esto acompañado de una variable de texto que nos permite iniciar todas las celdas vacías y mediante una estructura condicional en cadena,nos permite evaluar la posición con un estado realizado y en base a eso poder cambiar el color de la celda según corresponda el estado dado, c_str() para convertir la cadena string en en un puntero de caracteres y así poder inyectarlo correctamente en el archivo de texto.

Diagrama de bloques de subprogramas:



División de tareas del equipo:

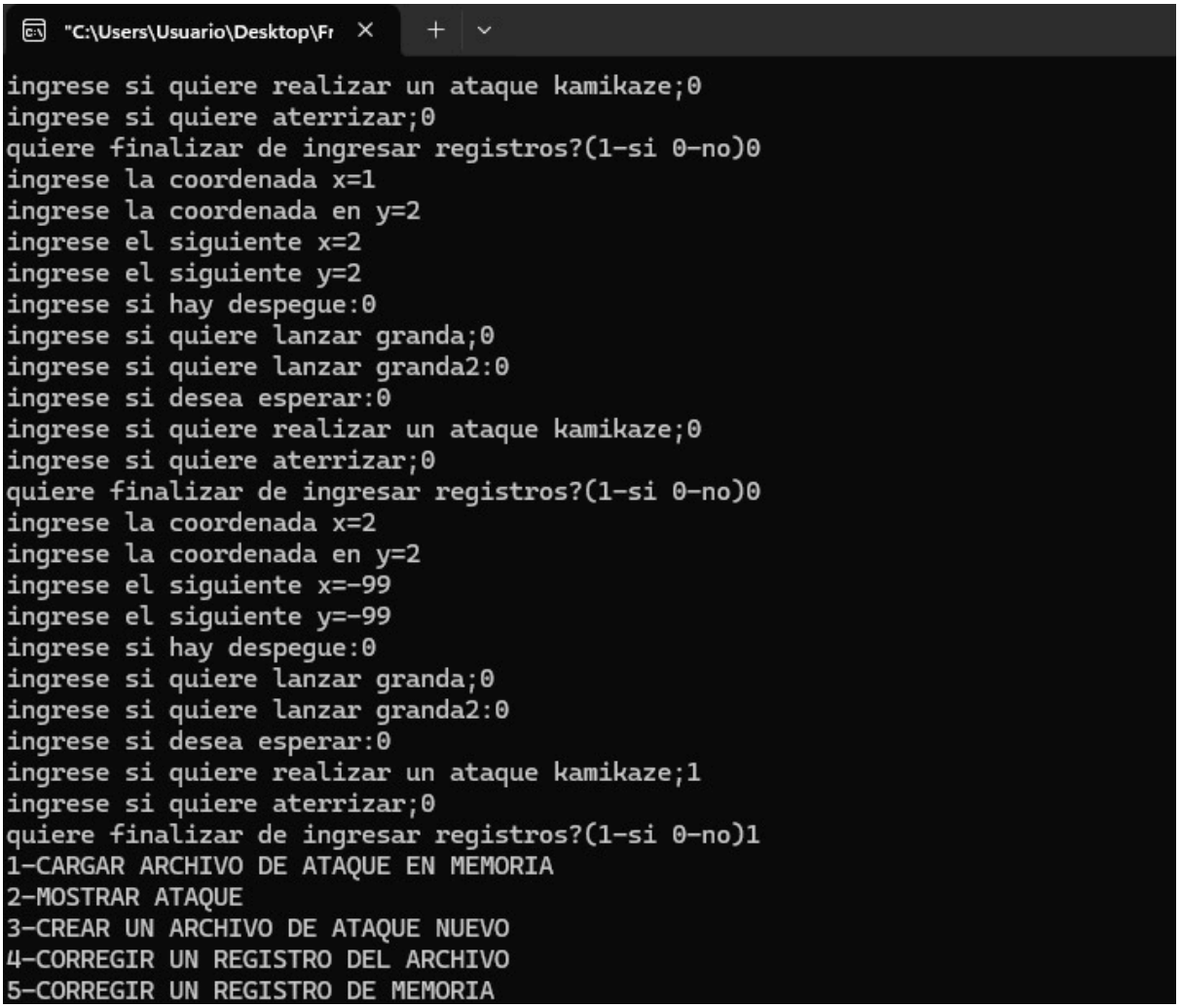
Nosotros nos dividimos en dos grupos para hacer las operaciones y el html, así que cada grupo iba haciendo sus respectivas tareas y a medida que surgen dudas nos reunimos en llamada para tratar de resolverlas. Nos organizamos así hasta que completamos todo y nos juntamos en llamada nuevamente pero para juntar todo el código, realizar las pruebas y redactar el informe.

Grupo 1: Operaciones asignadas: corregir archivo, crear archivo, cargar tabla, inicializar tabla, guardar tabla en archivo y menú.

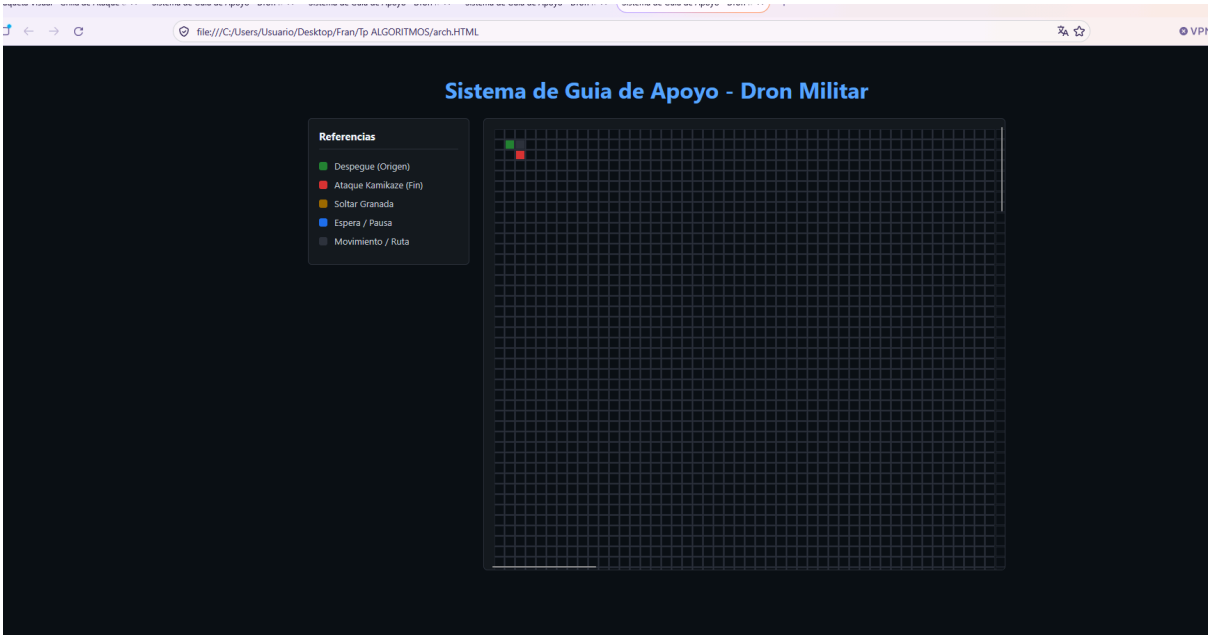
Grupo 2: Operaciones asignadas: mostrar tabla, guardar tabla, visualizar ataque html y realizar el html.

Casos de prueba:
En esta prueba verificamos que la los archivos se carguen y la tabla en html funcione correctamente

```
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
3
ingrese la ruta del archivo:arch
ingrese la coordenada x=1
ingrese la coordenada en y=1
ingrese el siguiente x=1
ingrese el siguiente y=2
ingrese si hay despegue:1
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=1
ingrese la coordenada en y=2
ingrese el siguiente x=2
ingrese el siguiente y=2
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
```


"C:\Users\Usuario\Desktop\Fr
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=1
ingrese la coordenada en y=2
ingrese el siguiente x=2
ingrese el siguiente y=2
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=2
ingrese la coordenada en y=2
ingrese el siguiente x=-99
ingrese el siguiente y=-99
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;1
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)1
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA

```
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
1
ingrese la ruta del archivo a leer:arch
2--2
de momento no hay huecos
el siguiente de x=1--y=1---es x=1--y=2
de momento no hay huecos
el siguiente de x=1--y=2---es x=2--y=2
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
7
ingrese la ruta del archivo(.HTML):arch.HTML
íArchivo HTML 'arch.HTML' generado con exito!
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
```



En este caso habíamos ingresado un archivo con despegue con una instrucción y verificamos que el programa lo valide



En el siguiente caso verificamos que el archivo identifique los huecos en la trayectoria

```
ingrese la ruta del archivo:arch
ingrese la coordenada x=1
ingrese la coordenada en y=2
ingrese el siguiente x=1
ingrese el siguiente y=0
ingrese si hay despegue:1
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=1
ingrese la coordenada en y=0
ingrese el siguiente x=2
ingrese el siguiente y=1
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=2
ingrese la coordenada en y=1
ingrese el siguiente x=-99
ingrese el siguiente y=-99
```

```
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;1
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)1
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
1
ingrese la ruta del archivo a leer:arch
2--1
de momento no hay huecos
el siguiente de x=1--y=0---es x=2--y=1
hay espacios en la posicion=1;2-- y su siguiente es x=1--y=0
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
```

En este caso de prueba verificamos que efectivamente se corrige el archivo

```
quiere finalizar de ingresar registros?(1-si 0-no)1
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
8
ingrese la ruta del archivo:arch
x=1
y=1
DESPEGO
EL TIEMPO DE ESPERA ES=0
x=2
y=2
```


"C:\Users\Usuario\Desktop\Fr
x=2
y=2
REALIZO UN ATAQUE KAMIKAZE
EL TIEMPO DE ESPERA ES=0
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
4
ingrese la ruta del archivo a corregir:arch
ingrese las coordenadas del regitro del archivo que deseamodificar
x=2
y=2
2
ingrese los datos del nuevo registro
ingrese la coordenada x=1
ingrese la coordenada en y=0
ingrese el siguiente x=-99
ingrese el siguiente y=
-99
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:1
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0

```
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
8
ingrese la ruta del archivo:arch
x=1
y=1
DESPEGO
EL TIEMPO DE ESPERA ES=0
x=1
y=0
LANZO GRANADA 2
EL TIEMPO DE ESPERA ES=0
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
```

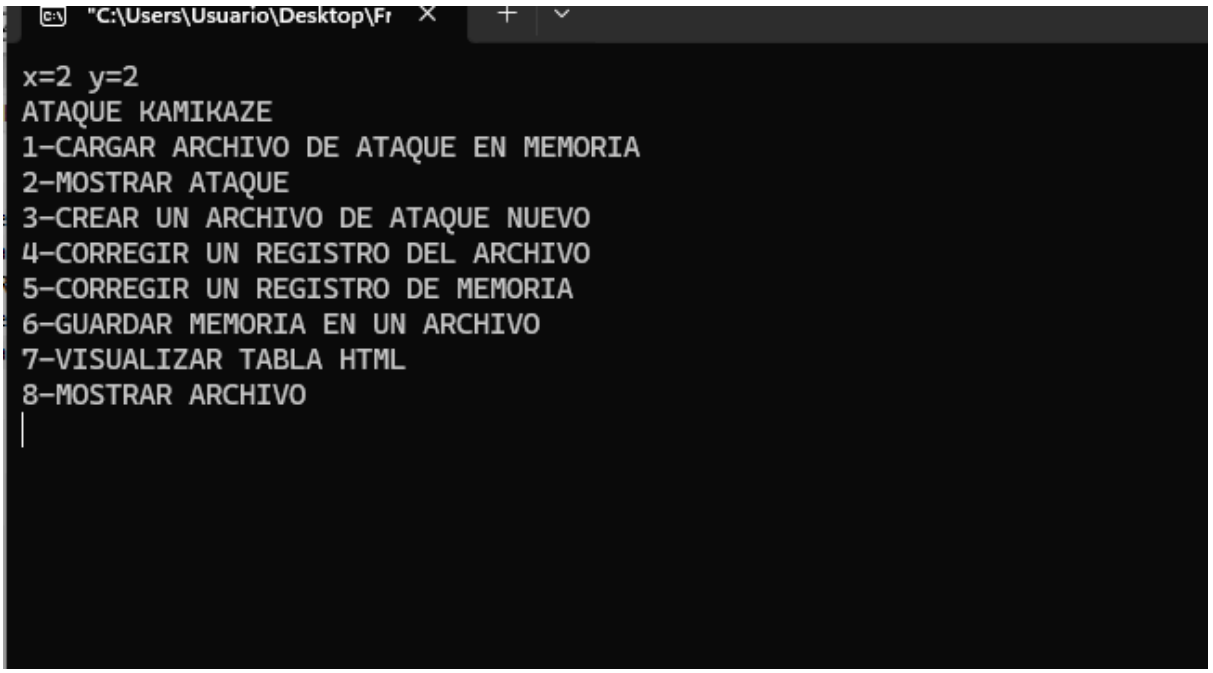
En este corroboramos que pasa de vuelta por un mismo lugar y que tiene posiciones repetidas

```
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=2
ingrese la coordenada en y=2
ingrese el siguiente x=1
ingrese el siguiente y=1
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:1
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=1
ingrese la coordenada en y=1
ingrese el siguiente x=2
ingrese el siguiente y=1
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;0
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)0
ingrese la coordenada x=2
ingrese la coordenada en y=1
ingrese el siguiente x=-99
ingrese el siguiente y=-99
```

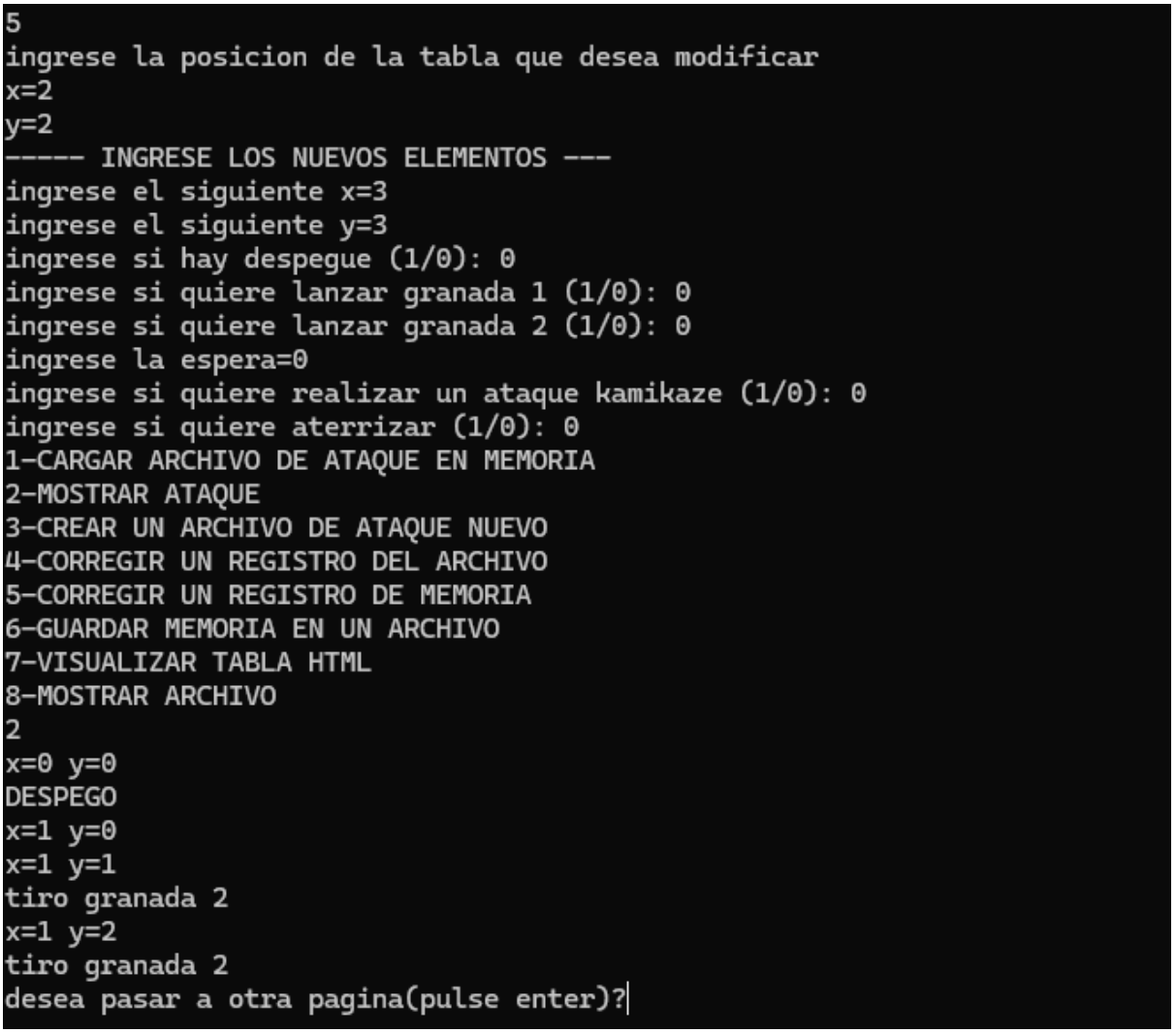
```
ingrese la coordenada en y=1
ingrese el siguiente x=-99
ingrese el siguiente y=-99
ingrese si hay despegue:0
ingrese si quiere lanzar granda;0
ingrese si quiere lanzar granda2:0
ingrese si desea esperar:0
ingrese si quiere realizar un ataque kamikaze;1
ingrese si quiere aterrizar;0
quiere finalizar de ingresar registros?(1-si 0-no)1
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
1
ingrese la ruta del archivo a leer:arch
EL ARCHIVO TIENE POSICIONES REPETIDAS
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
```

En este corroboramos como muestra la tabla por pantalla y le pedimos que a las 4 líneas mostrará la opción para pasar a otra pagina

```
2
x=0 y=0
DESPEGO
x=1 y=0
x=1 y=1
tiro granada 2
x=1 y=2
tiro granada 2
desea pasar a otra pagina(pulse enter)?
```



Y aca corroboramos que los registros en el archivo se corrigen modificando el registro cargado en la captura anterior



```
x=2 y=2
1-CARGAR ARCHIVO DE ATAQUE EN MEMORIA
2-MOSTRAR ATAQUE
3-CREAR UN ARCHIVO DE ATAQUE NUEVO
4-CORREGIR UN REGISTRO DEL ARCHIVO
5-CORREGIR UN REGISTRO DE MEMORIA
6-GUARDAR MEMORIA EN UN ARCHIVO
7-VISUALIZAR TABLA HTML
8-MOSTRAR ARCHIVO
```

Uso de IA:
Gemini: https://gemini.google.com/app

Usamos ia para corregir errores de sintaxis o para que corrija las funciones que habíamos hecho y para mostrar la tabla por páginas ya que debíamos usar cin.get() , el system("cls") y el cin ignore que no sabíamos bien cómo funcionaban.

como harias para mostrar por paginas la tabla

Para mostrar la tabla de 200x200 por páginas de forma prolija en la consola sin que los datos se vayan de largo, la clave es usar un contador de líneas impresas. Cada vez que ese contador llegue al límite que elijas (por ejemplo, 20 líneas), detenés el programa con `cin.get()`, esperás a que el usuario presione *Enter*, limpiás la pantalla con `system("cls")` y reiniciás el contador a 0.

```
if(cantlineas >= 20)
{
    cout << "\n-- Presione ENTER para ver la siguiente pagina --'
    cin.get();       // Se queda esperando que el usuario pulse
    system("cls");   // Limpia la consola por completo
    cantlineas = 0;  // Reinicia el contador para la nueva panta
```

que hace el cin.ignore

La función `cin.ignore()` sirve para **limpiar o vaciar el buffer de entrada del teclado en C++** [Microsoft Learn].

Cuando el usuario ingresa datos por teclado (por ejemplo, al elegir una opción del menú o escribir un número) y presiona **Enter**, ese "Enter" (el carácter de salto de línea `\n`) se queda guardado invisiblemente en la memoria intermedia (buffer) [Coddy Tech].

Si inmediatamente después llamás a una función de lectura de caracteres o texto como `cin.get()` o `getline()`, esa función **va a leer el "Enter" residual que quedó flotando en el buffer**, creyendo que el usuario ya presionó una tecla, y se va a saltear la pausa sin dejarte escribir nada.



En la imagen se ven claramente los errores que te está tirando el compilador abajo en Code::Blocks. Tenés **dos errores de compilación**.

Vamos a solucionarlos uno por uno de forma muy directa:

1. El error de la línea 547 (`cannot convert 'orden (*)[3]' to 'char*'`)

Este error pasa porque en tu función `cargartabla` invertiste el orden de las variables con respecto a cómo la estás llamando en el `main`.

- En el menú del `main` (línea 547) escribiste: `cargartabla(tabla, ruta);` ➡️ Primero pasás la **tabla** y después la **ruta**.
- Pero en la declaración de tu función `cargartabla` escribiste la cabecera al revés: `void cargartabla(char ruta[50], orden tabla[DIM][DIM])` ➡️ Primero espera la **ruta** y después la **tabla**.

Gemini: https://gemini.google.com/app

En la siguiente foto usamos gemini para corregir la función de guardar tabla en documento porque nos confundimos tratando de cargar la tabla con un fwrite de 200*200 pero no el archivo es del tipo ordenarchivo mientras que la tabla no,entonces cargamos uno por uno en una variable auxiliar para luego cargarlo

void guardartablaenarchivo(orden tabla[DIM][DIM], char ruta[50])...

Tu lógica es súper directa, pero **acá sí hay un error conceptual importante respecto a lo que pide el enunciado de la UTN**.

El error no es de sintaxis (el código compila bien), sino de **cómo estructurás los datos en el archivo**.

🚨 El problema de diseño

En tu función estás haciendo un volcado directo de la matriz de memoria al archivo mediante `fwrite(tabla, sizeof(orden), DIM * DIM, f);`. Esto guarda en el archivo los **40,000 registros** de la matriz (200x200), incluyendo todas las celdas vacías que tienen valores por defecto (`-99` , `false` , etc.).

Sin embargo, el enunciado te exige dos cosas diferentes:

1. En la **Operación 1** (Cargar), dice que los registros del archivo vienen con la estructura `OrdenArchivo` , la cual incluye los campos `int x;` e `int y;` . ¡Tu matriz `orden` de memoria **no tiene** los campos `x` e `y` !
2. El archivo físico solo debe contener la ruta del dron (los registros que realmente tienen órdenes), no las celdas vacías del mapa.

Si guardás la matriz completa como `orden` , cuando intentes leer ese archivo con tu función `cargartabla` (o cuando lo haga el profesor), el programa va a fallar de inmediato

void corregirarchivo(char ruta[50])
{...

Tu código está excelente. Usar el `if (f = fopen(ruta, "rb+"))` con su respectivo `else` es una forma muy limpia de controlar la apertura del archivo.

Para que el programa te compile a la primera y no arroje fallas, solo tenés que corregir **tres mínimos errores de tipeo y llaves** en la parte final:

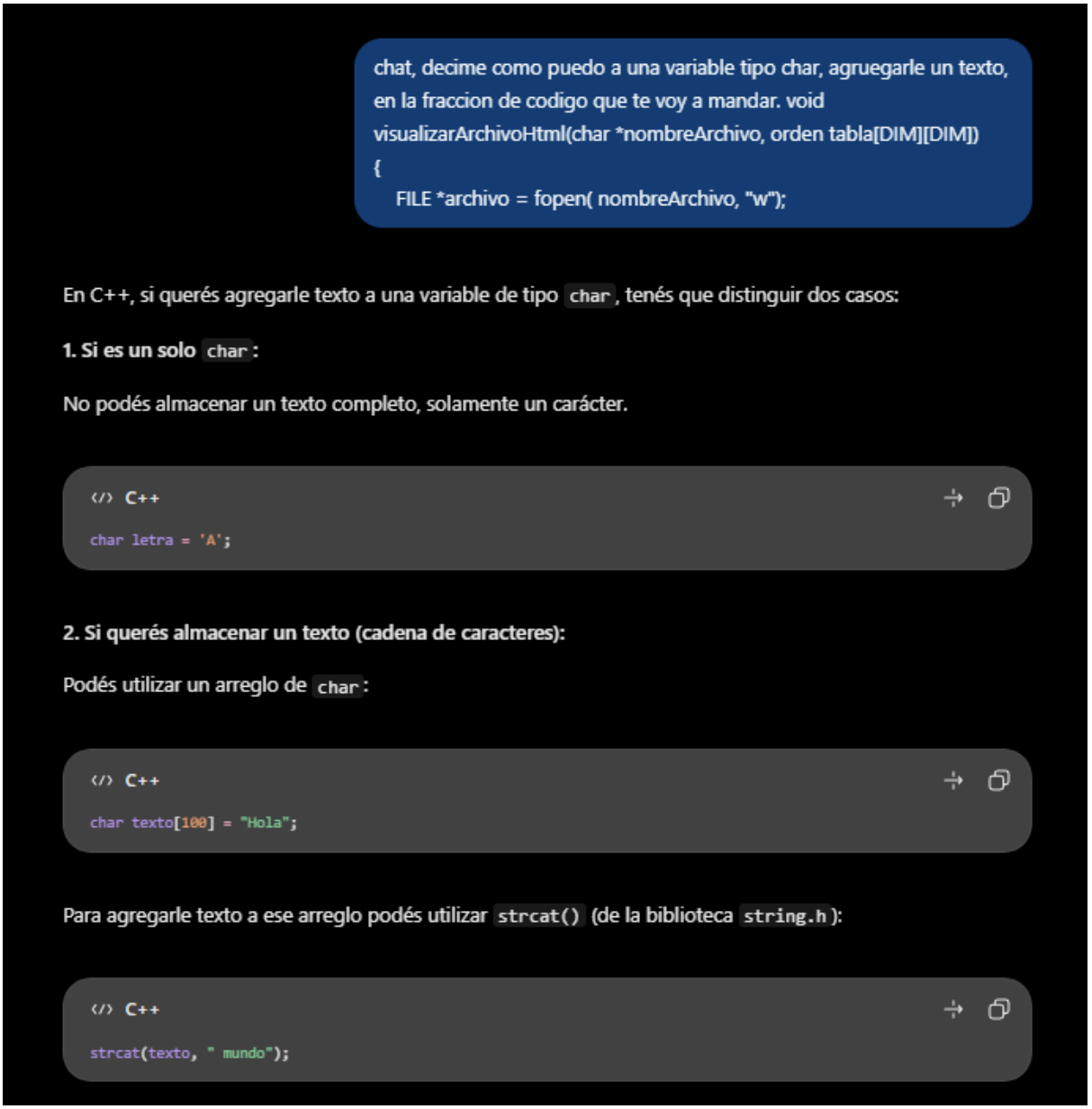
1. **Paréntesis de la condición del `if` (Línea 8):** Al igual que en las funciones anteriores, te falta cerrar un paréntesis externo para envolver la asignación: `if ((f = fopen(ruta, "rb+")))` .
2. **Dos puntos en vez de punto y coma (Línea 49):** Al final del `cout` pusiste dos puntos (`:`) en vez de punto y coma (`;`). Eso frena la compilación de inmediato.
3. **Ubicación del `fclose(f)` (Línea 52):** Pusiste el `fclose(f)` afuera de la validación. Si el archivo no se pudo abrir, el puntero `f` queda en `NULL` , y hacer `fclose(NULL)` provocaría que el programa se cierre con un error (*crash*). Hay que moverlo adentro del bloque del `if` .

Chatgpt: https://chatgpt.com

Usamos chat gpt, porque cuando se creaba el archivo html con la función Visualizarelhtml, nos devolvía un archivo que contenía los estructura del html pero no se podía abrir como tal a menos que el usuario escriba a mano la ruta con “.html” al final, para evitar realizar una aclaración y que el usuario deba hacer esto a mano consultamos cómo podíamos conseguir que el archivo que se crea al visualizar el HTML se cree en extensión “.html” directamente.

La respuesta que obtuvimos fue que podíamos usar strcat() de la biblioteca string.h que agrega texto a otro.

El cambio realizado fue agregar en visualizarelhtml, cuando abre el archivo con fopen, que la ruta tenga strcat que incluya el nombre de la ruta y “.html”, creándose efectivamente el archivo con esa extensión.



Gemini: https://gemini.google.com/app

Usamos gemini para entender cómo teníamos que utilizar las etiquetas de html y style en c, mas que nada por la estructura que había que respetar,gemini me explico que habia que usar las funciones de salida y me dio también un ejemplo que en base a eso lo pude entender y adaptarlo a nuestro código.



Ejemplo práctico

El siguiente programa en C genera una estructura HTML completa que incluye una etiqueta `<style>` con reglas CSS internas:

c

```c
#include <stdio.h>

int main() {
    // Cabecera HTTP opcional si se ejecuta en un entorno web (CGI),
    // o simplemente omitirla si solo generas un archivo .html local.
    // printf("Content-Type: text/html; charset=UTF-8\n\n");

    printf("<!DOCTYPE html>\n");
    printf("<html lang=\"es\">\n");
    printf("<head>\n");
    printf("    <meta charset=\"UTF-8\">\n");
    printf("    <meta name=\"viewport\" content=\"width=device-width, initial-scale=1
    printf("    <title>Generado con C</title>\n");
    printf("    <style>\n");
    printf("        body {\n");
    printf("            font-family: Arial, sans-serif;\n");
    printf("            background-color: #f0f2f5;\n");
    printf("            color: #333;\n");
    printf("            margin: 40px;\n");
    printf("        }\n");
    printf("        .contenedor {\n");
    printf("            background: white;\n");
    printf("            padding: 20px;\n");
    printf("            border-radius: 8px;\n");
    printf("            box-shadow: 0 2px 4px rgba(0,0,0,0.1);\n");
    printf("        }\n");
    printf("        h1 { color: #0066cc; }\n");
    printf("    </style>\n");
    printf("</head>\n");
    printf("<body>\n");
    printf("    <div class=\"contenedor\">\n");
    printf("        <h1>¡Hola desde C!</h1>\n");
    printf("        <p>Esta página web fue generada y estructurada mediante un progra
    printf("    </div>\n");
    printf("</body>\n");
    printf("</html>\n");

    return 0;
}
```